Merlette — API REST

- **Base URL** : `http://localhost:3000/api`
- **Format** : JSON (`Content-Type: application/json`), sauf upload (`multipart/form-data`).
- **Auth** : `Authorization: Bearer <token>` (JWT renvoyé par `/auth/login` ou `/auth/register`, valide 24 h).
- **Images** : servies statiquement sur `http://localhost:3000/uploads/<fichier>` .

Légende auth : 🌐 public · 🔒 connecté · 👑 admin · ✍️ auteur du contenu (ou admin).

1. Liste des endpoints

#	Méthode	URL	Auth	Description	Story
Auth					
1	POST	`/auth/register`	🌐	Créer un compte	#11
2	POST	`/auth/login`	🌐	Se connecter	#10
3	GET	`/auth/me`	🔒	Utilisateur courant	#10
Utilisateurs					
4	GET	`/users/:id`	🌐	Profil public + statistiques	#12
5	PATCH	`/users/me`	🔒	Modifier bio / avatar	#12
6	GET	`/users/me/observations`	🔒	Mes observations	#14
7	GET	`/users/me/conversations`	🔒	Mes discussions	#10
Référentiel					
8	GET	`/categories`	🌐	Liste des catégories	#9
9	POST	`/categories`	👑	Créer une catégorie	#20
10	GET	`/species?q=&category=`	🌐	Rechercher des espèces	#16
11	GET	`/species/:id`	🌐	Détail d'une espèce	#16
12	POST	`/species`	👑	Créer une espèce	#20
Observations					
13	GET	`/observations`	🌐	Fil paginé + filtres	#8, #9
14	POST	`/observations`	🔒	Publier (multipart, 1–5 photos)	#1, #13
15	GET	`/observations/:id`	🌐	Détail complet	#15
16	PATCH	`/observations/:id`	✍️	Modifier les infos	#14
17	DELETE	`/observations/:id`	✍️	Supprimer	#14, #19
18	POST	`/observations/:id/photos`	✍️	Ajouter des photos	#1

#	Méthode	URL	Auth	Description	Story
19	DELETE	`/observations/:id/photos/:photoId`	✍️	Retirer une photo (min. 1 restante)	#14
Identification					
20	GET	`/observations/:id/proposals`	🌐	Propositions + votes + votants	#5
21	POST	`/observations/:id/proposals`	🔒	Proposer une espèce (= vote)	#17
22	DELETE	`/proposals/:id`	✍️	Supprimer sa proposition (si 0 autre vote)	#17
23	PUT	`/observations/:id/vote`	🔒	Voter / changer de vote	#4
24	DELETE	`/observations/:id/vote`	🔒	Retirer son vote	#4
Commentaires					
25	GET	`/observations/:id/comments`	🌐	Liste chronologique	#3
26	POST	`/observations/:id/comments`	🔒	Commenter	#3
27	PATCH	`/comments/:id`	✍️	Modifier son commentaire	#18
28	DELETE	`/comments/:id`	✍️	Supprimer	#18, #19
Notifications					
29	GET	`/notifications?unread=true&page=`	🔒	Mes notifications	#6
30	GET	`/notifications/unread-count`	🔒	Badge (polling 30 s)	#6
31	PATCH	`/notifications/:id/read`	🔒	Marquer comme lue	#6
32	PATCH	`/notifications/read-all`	🔒	Tout marquer comme lu	#6
Divers					
33	GET	`/health`	🌐	Vérifier que l'API tourne	—

2. Format des erreurs

Toutes les erreurs ont la même forme :

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Les données envoyées sont invalides.",
    "details": [{ "field": "email", "message": "Email invalide" }]
  }
}
```

HTTP	code	Quand
400	`VALIDATION_ERROR`	Champ manquant ou invalide
401	`UNAUTHORIZED`	Token absent, invalide ou expiré
401	`INVALID_CREDENTIALS`	Login échoué
403	`FORBIDDEN`	Pas l'auteur / pas admin
404	`NOT_FOUND`	Ressource inexistante
409	`CONFLICT`	Email/pseudo déjà pris, espèce déjà proposée
413	`FILE_TOO_LARGE`	Image > 5 Mo
500	`INTERNAL_ERROR`	Erreur serveur

---

3. Détail des endpoints

Auth

`POST /auth/register`

```
// Requête
{ "username": "lea", "email": "lea@mail.com", "password": "Secret123" }

// 201 Created
{
  "token": "eyJhbGciOi...",
  "user": { "id": 2, "username": "lea", "email": "lea@mail.com", "role": "USER", "createdAt":
"2026-10-13T08:00:00Z" }
}
```

Règles : `username` 3–30 caractères `[a-zA-Z0-9_]`, `email` valide, `password` ≥ 8 caractères. Erreurs : 400, 409.

`POST /auth/login`

```
// Requête
{ "email": "lea@mail.com", "password": "Secret123" }
// 200 OK → même forme que register
```

Erreurs : 400, 401 `INVALID_CREDENTIALS`.

`GET /auth/me` 🔒

`200` → `{ "id": 2, "username": "lea", "email": "lea@mail.com", "role": "USER", "bio": null, "avatarUrl": null }`

Utilisateurs

`GET /users/:id`

```
{
  "id": 2, "username": "lea", "bio": "Randonneuse", "avatarUrl": null, "createdAt": "...",
  "stats": { "observations": 12, "proposals": 4, "identifiedObservations": 7 }
}
```

`PATCH /users/me` 🔒

`{ "bio": "...", "avatarUrl": "..." }` → `200` utilisateur mis à jour.

`GET /users/me/observations` 🔒

Même format que `GET /observations`.

`GET /users/me/conversations` 🔒

```
{
  "data": [
    { "observationId": 42, "title": "Petit oiseau", "status": "NEEDS_ID", "coverUrl":
"/uploads/a.jpg",
      "lastActivity": "2026-10-13T09:00:00Z", "myRole": ["AUTHOR", "COMMENTER"] }
  ]
}
```

Référentiel

`GET /categories`

`200` → `[{ "id": 2, "name": "Oiseaux", "slug": "oiseaux" }, ...]`

`POST /categories` 👑

`{ "name": "Lichens", "slug": "lichens" }` → `201`. Erreurs : 403, 409.

`GET /species?q=mesange&category=oiseaux&limit=10`

Recherche insensible à la casse et aux accents sur `common_name` et `scientific_name` (min. 2 caractères).

```
[{ "id": 1, "commonName": "Mésange bleue", "scientificName": "Cyanistes caeruleus", "category": {
"id": 2, "slug": "oiseaux" } }]
```

`POST /species` 👑

`{ "commonName": "Sittelle torchepot", "scientificName": "Sitta europaea", "categoryId": 2 }` → `201`.

Observations

`GET /observations`

Paramètre	Type	Défaut	Exemple
`category`	slug	toutes	`oiseaux`
`status`	`NEEDS_ID` | `IDENTIFIED`	tous	`NEEDS_ID`
`userId`	int	—	`2`
`sort`	`recent` | `oldest`	`recent`	
`page`	int ≥ 1	1	
`limit`	int 1–50	20	


```
{
  "data": [
    {
      "id": 42, "title": "Petit oiseau", "status": "NEEDS_ID",
      "category": { "id": 2, "name": "Oiseaux", "slug": "oiseaux" },
      "author": { "id": 2, "username": "lea" },
      "coverUrl": "/uploads/1697-a.jpg",
      "observedAt": "2026-10-10", "createdAt": "2026-10-13T08:10:00Z",
      "proposalCount": 2, "commentCount": 3,
      "identifiedSpecies": null
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 57, "totalPages": 3 }
}
```


`POST /observations` 🔒 — **`multipart/form-data`**

Champ	Type	Obligatoire
`photos`	fichier(s), 1–5, JPEG/PNG/WEBP, ≤ 5 Mo	✅
`categoryId`	int	✅
`observedAt`	date `YYYY-MM-DD`, ≤ aujourd'hui	✅
`title`	string ≤ 120	
`description`	string	
`locationName`	string ≤ 200	
`latitude` / `longitude`	nombre	

`201` → observation complète (format de `GET /observations/:id`). Erreurs : 400, 401, 413.

`GET /observations/:id`

```
{
  "id": 42, "title": "Petit oiseau", "description": "Sur la mangeoire",
  "status": "IDENTIFIED",
  "identifiedSpecies": { "id": 1, "commonName": "Mésange bleue", "scientificName": "Cyanistes caeruleus" },
  "category": { "id": 2, "name": "Oiseaux", "slug": "oiseaux" },
  "author": { "id": 2, "username": "lea" },
  "observedAt": "2026-10-10", "locationName": "Parc de la Tête d'Or", "latitude": 45.7772, "longitude": 4.8551,
  "photos": [{ "id": 1, "url": "/uploads/1697-a.jpg", "position": 0 }],
  "proposals": [ /* même format que GET /observations/:id/proposals */ ],
  "myVoteProposalId": 5,
  "commentCount": 3,
  "createdAt": "...", "updatedAt": "..."
}
```

`myVoteProposalId` vaut `null` si non connecté ou pas de vote.

`PATCH /observations/:id` ✍️

JSON avec les champs modifiables (`title` , `description` , `categoryId` , `observedAt` , `locationName` , `latitude` , `longitude`). `200` . Erreurs : 400, 403, 404.

`DELETE /observations/:id` ✍️ **/** 👑

`204 No Content` . Supprime en cascade photos, propositions, votes, commentaires, notifications et fichiers.

`POST /observations/:id/photos` ✍️

Multipart `photos` . Erreur 400 si le total dépasse 5.

`DELETE /observations/:id/photos/:photoId` ✍️

`204` . Erreur 400 si c'est la dernière photo.

Identification

`GET /observations/:id/proposals`

```
{
  "totalVotes": 4,
  "consensus": { "reached": true, "proposalId": 5, "speciesId": 1 },
  "proposals": [
    {
      "id": 5, "species": { "id": 1, "commonName": "Mésange bleue", "scientificName": "Cyanistes caeruleus" },
      "proposedBy": { "id": 3, "username": "karim" },
      "votes": 3, "percent": 75.0,
      "voters": [{ "id": 3, "username": "karim" }, { "id": 4, "username": "sam" }, { "id": 2, "username": "lea" }],
      "createdAt": "..."
    },
    { "id": 6, "species": { "id": 2, "commonName": "Mésange charbonnière", "scientificName": "Parus major" },
      "proposedBy": { "id": 5, "username": "tom" }, "votes": 1, "percent": 25.0, "voters": [ { "id": 5, "username": "tom" } ], "createdAt": "..." }
  ]
}
```

`POST /observations/:id/proposals` 🔒

```
// Requête
{ "speciesId": 1 }
// 201 Created
{ "proposal": { "id": 5, "speciesId": 1, "votes": 1 }, "observationStatus": "NEEDS_ID" }
```

Effets : crée la proposition, place/déplace le vote de l'utilisateur dessus, notifie l'auteur, recalcule le consensus. Erreurs : 400 (`speciesId` manquant), 404 (observation ou espèce inexistante), 409 (espèce déjà proposée → voter avec `PUT /vote`).

`DELETE /proposals/:id` ✍️

`204`. Erreur 409 si d'autres utilisateurs ont voté pour elle.

`PUT /observations/:id/vote` 🔒

```
// Requête
{ "proposalId": 5 }
// 200 OK
{ "myVoteProposalId": 5, "observationStatus": "IDENTIFIED", "identifiedSpecies": { "id": 1, "commonName": "Mésange bleue" } }
```

Idempotent : revoter pour la même proposition ne change rien. Erreurs : 400, 401, 404 (proposition n'appartenant pas à cette observation).

`DELETE /observations/:id/vote` 🔒

`200` → `{ "myVoteProposalId": null, "observationStatus": "NEEDS_ID", "identifiedSpecies": null }`. Erreur 404 si aucun vote.

Commentaires

`GET /observations/:id/comments`

```
[{ "id": 9, "content": "Calotte bleue → mésange bleue", "author": { "id": 3, "username": "karim" },
   "createdAt": "...", "updatedAt": "...", "edited": false }]
```

`POST /observations/:id/comments` 🔒

`{ "content": "..." }` (1–1000 caractères) → `201` commentaire créé. Erreurs : 400, 401, 404.

`PATCH /comments/:id` ✍️

`{ "content": "..." }` → `200`, `edited: true`. Erreurs : 400, 403, 404.

`DELETE /comments/:id` ✍️ / 👑

`204`.

Notifications

`GET /notifications?unread=true&page=1`

```
{
  "data": [
    { "id": 15, "type": "OBSERVATION_IDENTIFIED", "message": "Votre observation a été identifiée : Mésange bleue",
      "observationId": 42, "actor": { "id": 3, "username": "karim" }, "isRead": false, "createdAt": "..." }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

`GET /notifications/unread-count`

`{ "count": 2 }`

`PATCH /notifications/:id/read`

`200` → notification avec `isRead: true`. Erreur 404 si elle n'appartient pas à l'utilisateur.

`PATCH /notifications/read-all`

`200` → `{ "updated": 5 }`

---

4. Structure back-end suggérée

```
server/
├── src/
│   ├── app.js                # Express, CORS, JSON, /uploads statique, routes, erreurs
│   ├── config.js             # lecture .env (PORT, DATABASE_URL, JWT_SECRET)
│   ├── db.js                 # pool pg / client Prisma
│   ├── middlewares/
│   │   ├── auth.js           # requireAuth, optionalAuth, requireAdmin
│   │   ├── upload.js         # Multer (types, taille, 5 fichiers max)
│   │   └── errorHandler.js   # format d'erreur uniforme
│   ├── routes/               # auth, users, categories, species, observations, proposals,
comments, notifications
│   └── services/
│       ├── consensus.service.js     # règle §6.1 du PRD
│       └── notification.service.js  # §6.2 du PRD
├── uploads/
└── .env
```